

기능 하나 고치는데, 이 로직은 어디에 뒀야 할까요?



백엔드 코드를 고치다 보면 비슷한 고민이 반복됩니다.

역할을 나누기 시작하면, 상태의 흐름이 보입니다.

Request → UseCase → Engine → Repository



처음에는 백엔드 계층의 역할을 어떻게 나눌지 고민했습니다. 그런데 상태를 누가 복원하고, 계산하고, 기록하는지 구분하기 시작하면서 동시성 문제까지 이어졌습니다.

UseCase는 흐름을, Domain은 규칙을 봅니다.

UseCase = 흐름 · Domain = 규칙



Domain Test는 State에서 Rule을 거쳐 Result가 맞는지를 봅니다. UseCase Test는 Read, Orchestrate, Write, Side Effect의 흐름을 봅니다.

상태를 누가 읽고, 계산하고, 기록할까요?

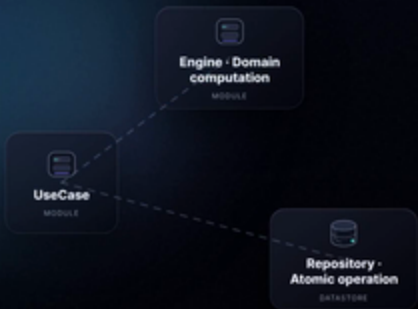
Restore → Compute → Record



Repository가 상태를 복원하고, UseCase가 흐름을
조율하고, Domain이 계산하고, 다시 Repository가
기록하도록 책임을 나눴습니다.

Engine은 필수 계층이 아니라, UseCase가 선택하는 도구입니다.

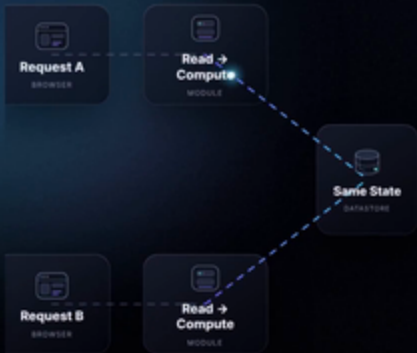
필요할 때만 Engine



Engine은 반드시 통과해야 하는 계층이 아니었습니다. Domain computation이 필요할 때 UseCase가 선택하는 도구로 보는 편이 자연스러웠습니다.

순수한 Domain 바깥에는 동시성의 창이 생깁니다.

Read → Compute → Write



Domain을 순수하게 만들면 테스트는 쉬워집니다. 하지만 상태를 읽고 다시 기록하는 사이에 다른 요청이 끼어들 수 있습니다.

단순한 경쟁은 원자적 연산으로 줄일 수 있습니다.

Repository · tryDecrease()



경쟁 조건이 핵심인 단순 변경이라면 Repository의 atomic operation을 직접 호출하는 편이 더 자연스러울 수 있습니다.

원자화를 너무 밀면, 책임이 다 시 흐려질 수 있습니다.

동시성 단순화 ↔ 책임 집중



반대로 모든 변경을 Repository의 atomic SQL로
밀어 넣으면 동시성은 단순해질 수 있지만,
Persistence Layer가 Application의 비즈니스 로
직을 흡수하기 시작합니다.

설계의 기준은 계층 수가 아니라 경계입니다.



결국 계층을 더 만드는 게 핵심은 아닙니다.

BLOG / CONTINUE

**더 자세한 이야기는
블로그에서 보실 수 있습니다.**

블로그에서 읽기

blog.dohyeon.kr

